

Storage migration: per-row Postgres tables + horizontal scaling (Redis)

PR **#5** github.com/dani3l-1/LinkUp/pull/5Branch `feat/horizontal-scale` → `main`Changes **+1,342** **-66** across 5 files · 9 commits

Description

What this does

Moves LinkUp's storage off the single `linkup_state` JSON blob into **per-row Postgres tables**, and adds the Redis-backed shared state needed to run **more than one app instance**. Done on production's current code — every production feature (safety, moderation, mobile tokens, feedback, ride-alerts, Events, boarding pass, etc.) is preserved.

Before: every write deep-cloned and rewrote the whole database (measured 4ms→245ms as the DB grew to 20k users), and two instances would clobber each other's writes. After: a write touches one indexed row, and instances stay coherent via Redis.

Entity migration (8 tables)

`linkup_users`, `linkup_rides`, `linkup_ride_requests`, `linkup_messages`, `linkup_payments`, `linkup_wallet_transactions`, `linkup_checkout_sessions`, `linkup_tracking_trips`.

Mechanism (`MIGRATED_BLOB_KEYS`): `loadDb()` repopulates `db.<entity>` from each store's cache so the ~200 read sites are untouched; `saveDb()` strips those keys from the blob write and runs change-detection; high-write paths reroute to per-row `persist / append`; rides + users + money records use signature-based change-detection so no in-place mutation site can be missed.

Horizontal scaling (inert without `REDIS_URL`)

- Shared **rate limiter** (Redis fixed-window, in-memory fallback)
- **Socket.IO Redis adapter** (cross-instance chat / live-trip fan-out)
- **Cross-instance cache invalidation** bus (every committed write publishes; peers reload that one row)
- **DB-enforced money idempotency**: partial `UNIQUE` index on `linkup_wallet_transactions(dedupe_key)` + `insertUnique` — a double driver-credit is impossible even if two instances race the same checkout
- `APP_INSTANCE_COUNT` guard relaxed to allow >1 **only when** `REDIS_URL` **is set**, plus a connection-budget guard

Two pre-existing production bugs fixed

- `statement_descriptor` on card `PaymentIntents` — Stripe rejects this; it breaks *every* card checkout in production today. Switched to `statement_descriptor_suffix`.

- **Backfill ordering data-loss bug** — each store's one-time backfill called `saveDb`, which strips all migrated keys, so on a real deploy the first store to load would wipe the other entities' legacy blob data before they could migrate. Fixed to strip once, after all backfills.

Verification (real throwaway Postgres + Stripe test mode + Upstash Redis)

- **Full Stripe-test checkout — 12/12:** payment, the driver's **\$16.12** credit (net of Stripe fee + 15% commission), and the paid session each land in their own table; blob carries zero; seat reserved on the ride row; a duplicate credit inserts **0 rows** (count stays 1).
- **Backfill-no-loss — 5/5:** a blob seeded with all 8 entity types migrates every one; blob left clean (regression test for the ordering bug).
- **Two-instance coherence — 9/9:** user / ride / message / profile edits propagate both directions across two instances behind one Redis.
- `npm test` green at every commit; branch boots at every commit.

⚠ Before deploying

- Existing data migrates automatically out of the blob on first boot (one-time, idempotent).
- Production stays **single-instance** until you explicitly set `APP_INSTANCE_COUNT>1` (with `REDIS_URL` set and `DATABASE_POOL_MAX` sized so the product fits the DB pooler).
- The unmerged **Social / Bites** layer is a **separate** effort — not in this PR.

🤖 Generated with [Claude Code](#)

Commits (9)

	SHA	MESSAGE
1	c3b70a5	Scale infra: Redis rate limiter + Socket.IO adapter + invalidation bus
2	81497ff	Migrate tracking trips to per-row table (linkup_tracking_trips)
3	05b18cc	Migrate ride-requests to per-row table (linkup_ride_requests)
4	7a7b7eb	Migrate rides to per-row table (linkup_rides)
5	b60af86	Migrate ride chat to per-row table (linkup_messages)
6	079693d	Migrate payments, checkout sessions, wallet to per-row tables (money)
7	2a21ef4	Migrate users to per-row table + fix backfill ordering data-loss bug
8	58c614c	launch-check: read deleted user from users.json (users are now per-row)
9	84b2c92	docs: add scalability roadmap + architecture diagram for the migration

Files changed

docs/operations/architecture.md		152	+++++
docs/operations/scalability.md		247	+++++++
package.json		2	+
scripts/check-launch-readiness.js		6	+−

```
server.js | 1001 ++++++-----  
5 files changed, 1342 insertions(+), 66 deletions(-)
```

Generated 2026-07-19 from PR #5. Not for production until reviewed, merged, and deployed per the runbook.